# poisson-solver Performance Evaluation

Matous Zatloukal

## Problem Description

### Motivation behind `poisson-solver`

`PoisFFT` is a Fortran library that solves the Poisson equation using the Fast Fourier Transform [1]. It is a part of ELMM (Extended Large Eddy Microscale Model), a LES (Large Eddy Simulation) model for atmospheric flows.

`PoisFFT` is written entirely in Fortran and utilizes only CPUs. It is parallelized using `OpenMP` in a single-node (local) configuration and using `MPI` in distributed configuration. Furthermore, it offers a hybrid `MPI`/`OpenMP` parallelization in distributed configurations. `PoisFFT` requires the users to first define a domain (number of grid cells, grid lengths, boundary conditions, etc.) and then allows them to repeatedly solve the Poisson equation with this domain configuration by supplying the right-hand side of the equation $\Delta\phi = b$.

Equation solving can be summarized as the following pipeline:

1. Apply a forward FFT on the whole RHS,
2. divide each RHS element by a sum of pre-computed eigenvalues,
3. apply a backward FFT on the whole RHS and
4. normalize the result to get $\Phi$.

For different combinations of BCs (boundary conditions) the pipeline may slightly vary (e.g., the real RHS is first converted into a complex RHS or different types of FFTs are applied separately to 1D pencils in one direction and then to 2D slabs in the other directions) For distributed solvers it is more complicated due to the requirements of data locality for FFT libraries. However, the core steps are still present. Apart from the FFT steps, `PoisFFT` is a 1-point stencil computation which makes it a great candidate for a rewrite into C++ and CUDA to take advantage of the GPUs available on the MFF cluster `chimera`.

`poisson-solver` is exactly this rewrite which we've done as a part of Research Project (NPRG070). As stated before, it utilizes GPUs which are more suited for this type of a problem. For the distributed communication it also utilizes `MPI`.

### Local Solvers Evaluation

For non-distributed solvers, we want to determine the speedup of `poisson-solver` compared to `PoisFFT` in all possible combinations of:

1. Precision (double or float),
2. dimensionality of the grid (1, 2 or 3),
3. uniformity of the grid on the Z axis and
   - Both solvers support the option to have the grid cell sizes nonuniform in the Z axis, this requires a different solving method.
4. boundary conditions (dependent on the dimensionality and uniformity of Z axis).

In addition to those, `PoisFFT` will also be measured using different combination of cores (16, 32 and 48).

The grid side sizes will range from 64 to 512 with the step of 64.

We suspect that the buffer copying between host and device will be the bottleneck in `poisson-solver`. Therefore, `poisson-solver` provides additional API that accepts device buffers and copies buffers only between devices. We also want to do the previous measurements using this API to simulate the situation where other parts of `ELMM` are rewritten and the inputs for `poisson-solver` are already on the device.

To help us determine the bottleneck of the pipeline for further research focus, we will measure the following in `poisson-solver`:

1. Time spent copying buffers,
2. time spent executing FFTs,
3. time spent on our kernel execution (e.g., division by eigenvalues) and
4. total time spent solving the equation.

Only the total time spent solving the equation will be measured for `PoisFFT`. We cannot measure the individual metrics as `PoisFFT` has no buffer copying and the FFTs are sometimes interleaved with kernel execution inside `OpenMP` parallel blocks.

### Distributed Solvers Evaluation

For distributed solvers, we will measure the same metrics as for local solvers. We will measure a subset of the configurations. That is strong scaling on 3D grids of side size 512 and weak scaling on 3D grids with local side size 256. Distributed solvers won't realistically be run on smaller problems and therefore doesn't make sense to measure them in those scenarios.

### ELMM Evaluation

TODO - How to measure in ELMM?

- ELMM already outputs the total time to execute a simulation and time spent in Poisson solver
  - It's very bare-bones but maybe it can be used?

## Benchmark Descriptions

The benchmark source code and runner scripts for `poisson-solver` can be found in the `benchmarks` branch of `poisson-solver`. The benchmark source code and runner scripts for `PoisFFT` can be found in my `PoisFFT` fork.

It is important to note that `PoisFFT` was modified for these benchmarks. During the initial analysis, we found that `PoisFFT` wasn't correctly linking and afterwards initializing `fftwf`. This resulted in slower execution time for `PoisFFT` in `float` precision configurations. The linking fix can be found here and the initialization fix here.

### Local Solvers

The source code for `poisson-solver` is in `poisson-solver/tests/benchmarks/benchmark.cpp` and for `PoisFFT` in `PoisFFT/src/benchmark.f90`. The runner script for `poisson-solver` is in `poisson-solver/local_benchmark.sh` and for `PoisFFT` in `PoisFFT/local_benchmark.sh` (both have their `chimera` SLURM launchers in `run_local_benchmark.sh`).

The runner scripts loop through all the aforementioned combinations of configurations and for each one launch the benchmark. The benchmark then loops over all given grid sizes and for each one constructs the solver and repeatedly measures the aforementioned metrics of one iteration (for given amount of iterations).

#### poisson-solver

The total time is measured using `std::steady_clock` as this clock is guaranteed to be monotonic (unlike `std::high_resolution_clock` which may be aliased to non-monotonic `std::system_clock`). The buffer

copy time and time to execute our own kernels is measured using `CUDAScopedTimer` and everything else using `HybridScopedTimer`. Both scoped timers accept a reference in their constructor into which they accumulate the measured time. Both also start the measurement in the constructor and stop it in the destructor.

`CUDAScopedTimer` uses CUDA events to measure the time of operations submitted into the CUDA stream (i.e., strictly GPU-related operations).

`HybridScopedTimer` uses `cudaDeviceSynchronize()` along with `std::steady_clock` to measure the time of operations executed by both CPU and GPU. By calling `cudaDeviceSynchronize()` before starting and then before stopping the measurement, we ensure we measure only the GPU kernels and all other CPU operations in the given scope. The `HybridScopedTimer` is utilized for measurements where we can't guarantee only CPU or only GPU usage and need to therefore measure both (e.g., calls to an external library).

#### PoisFFT

The total time is measured using `SYSTEM_CLOCK`.

### Distributed Solvers

TODO

## Platform

The local solver benchmarks were compiled and executed on the `bw01` node on `chimera` cluster. `bw01` has: 2x NVIDIA RTX PRO Blackwell Server Edition, PCIe 96 GB (only one is used for local benchmarks). The CPU on `bw01` is an AMD EPYC 9454 48-Core Processor supporting avx, avx2, avx512 vector ISA. The CUDA version on `bw01` is 13.3.

TODO - Describe environment on voltas

The distributed solver benchmarks were executed on nodes `volta02`, `volta03` and `volta05`.

## Analysis

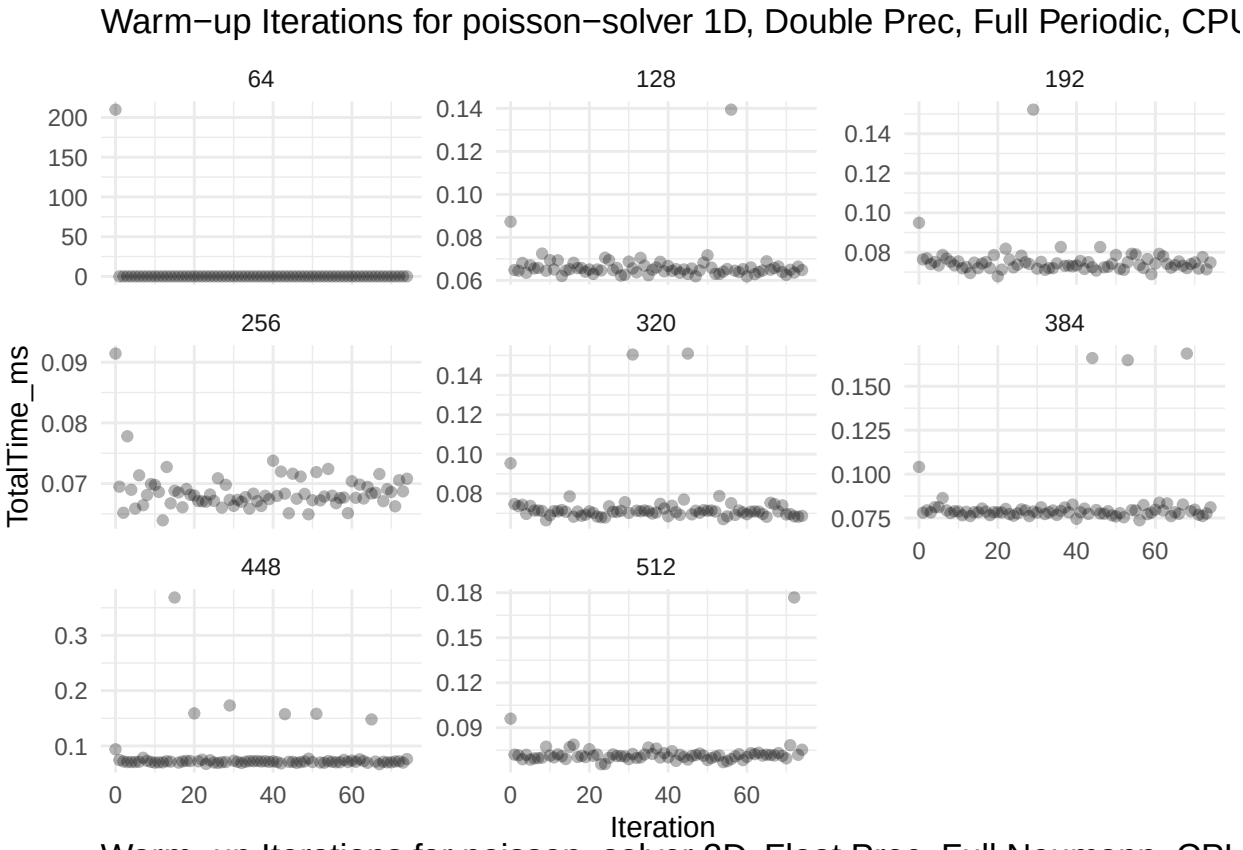
### Local Solvers

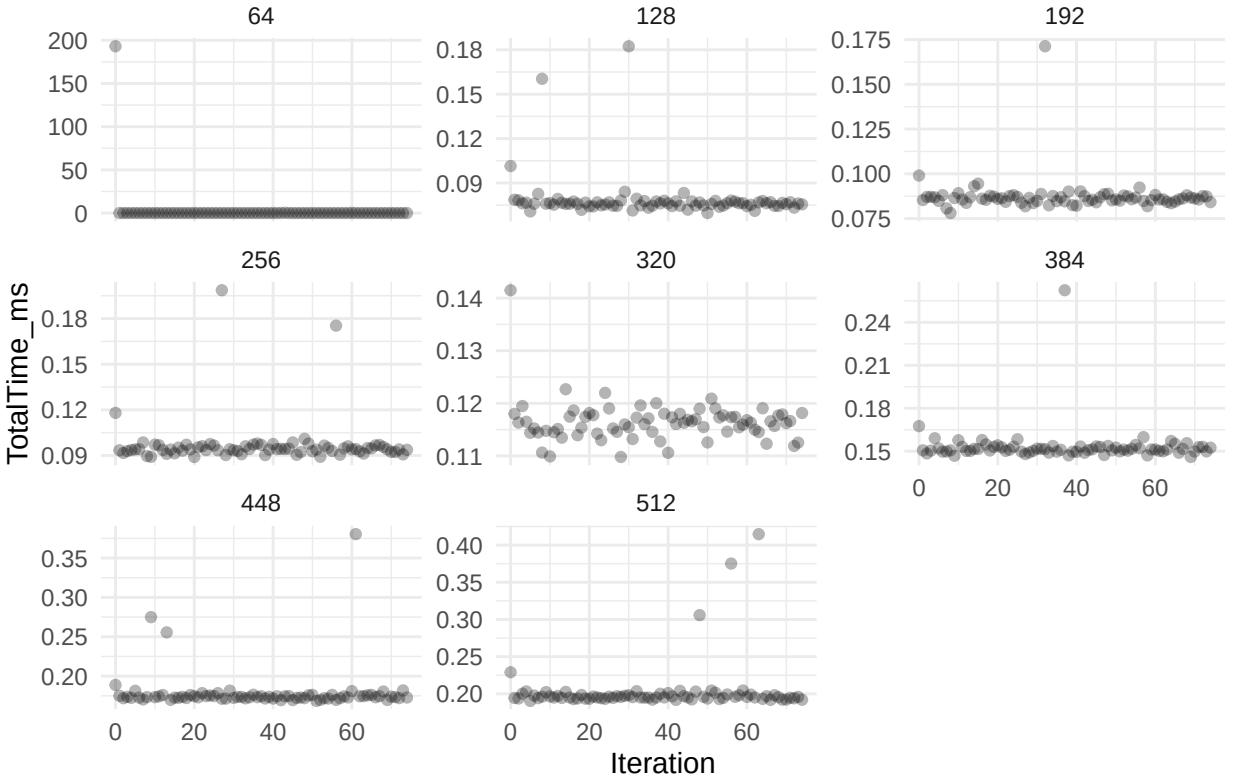
#### Warm-up and Outliers

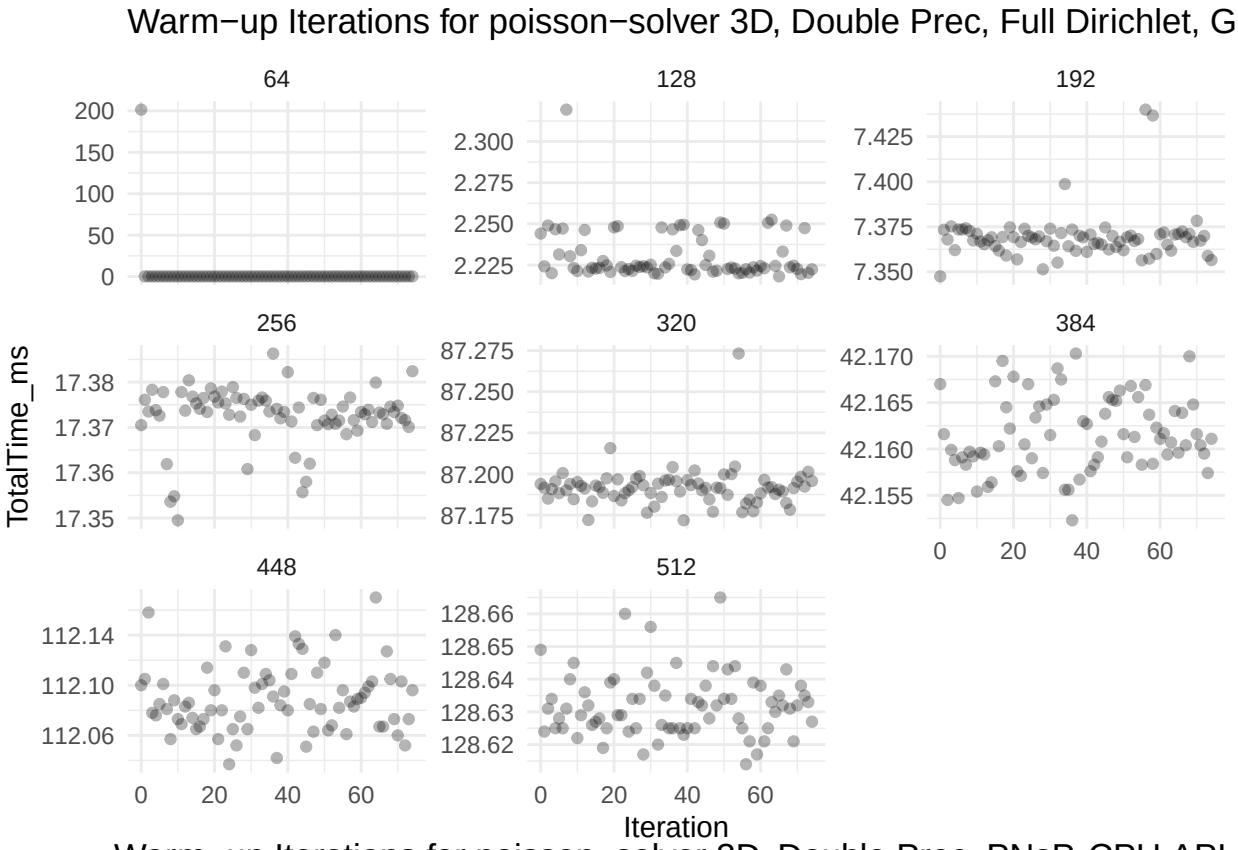
First, we will determine the amount of iterations until warm-up finishes and discard those. Due to how few measurements we have of one configuration, we will do this manually for pre-selected configurations and determine the number of warm-up iterations from those. The number of warm-up iterations shouldn't not differ between different solver configurations.

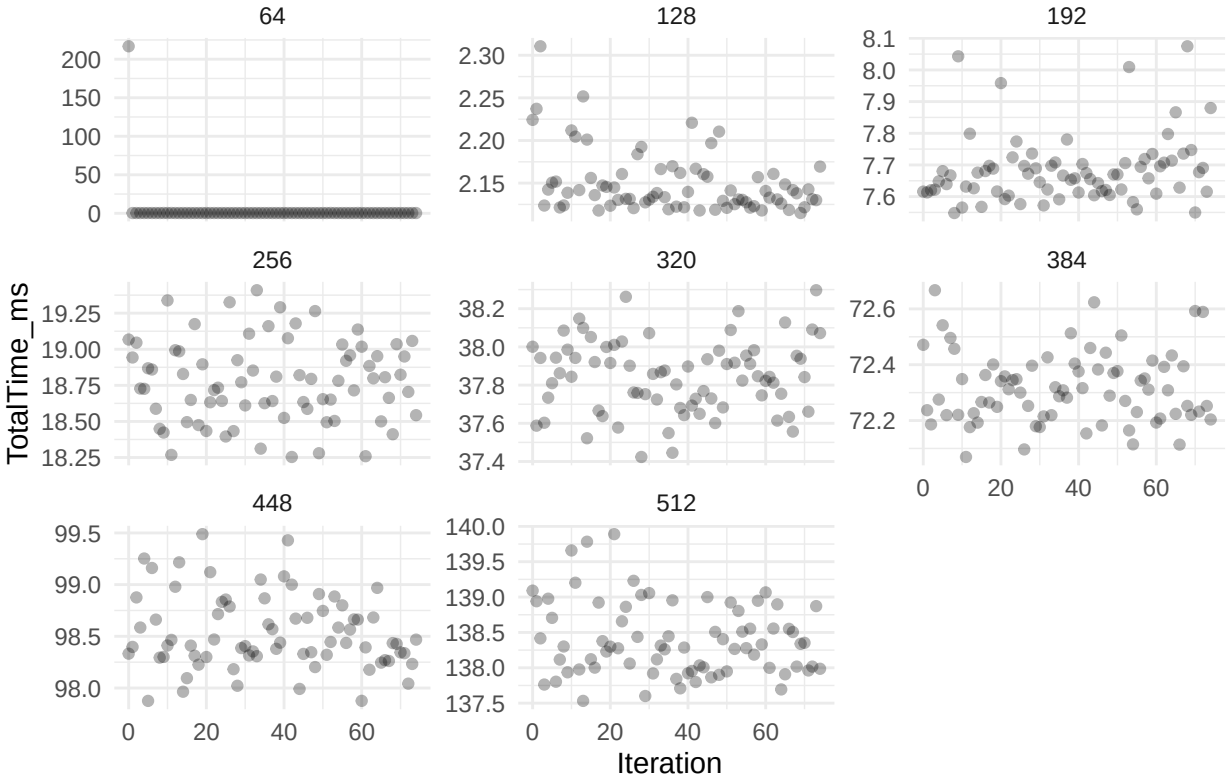
Afterwards, we will determine if there are any outliers and discord those as well.

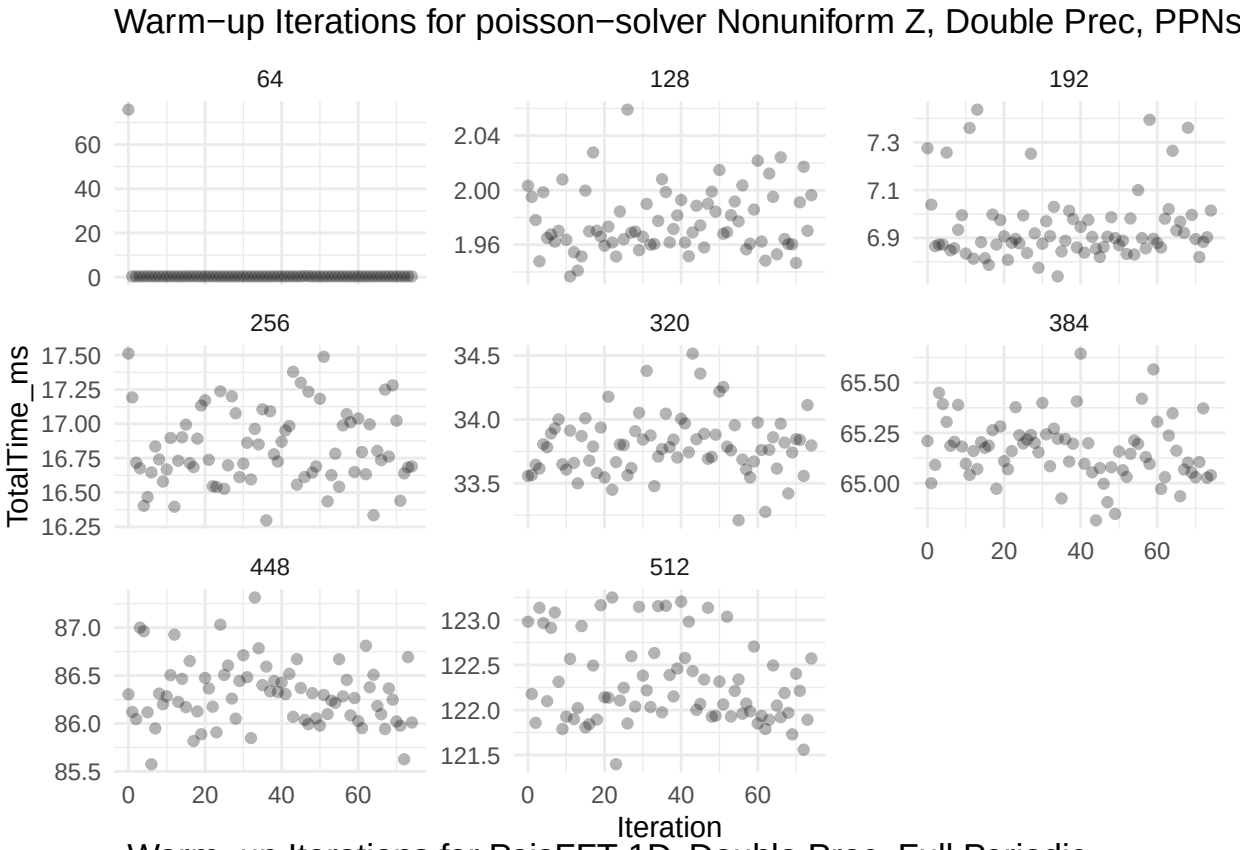
We expect there to be a one or two iterations long warm-up for `poisson-solver`. There may be more for both `PoisFFT` and `poisson-solver` if FFT libraries perform autotuning.

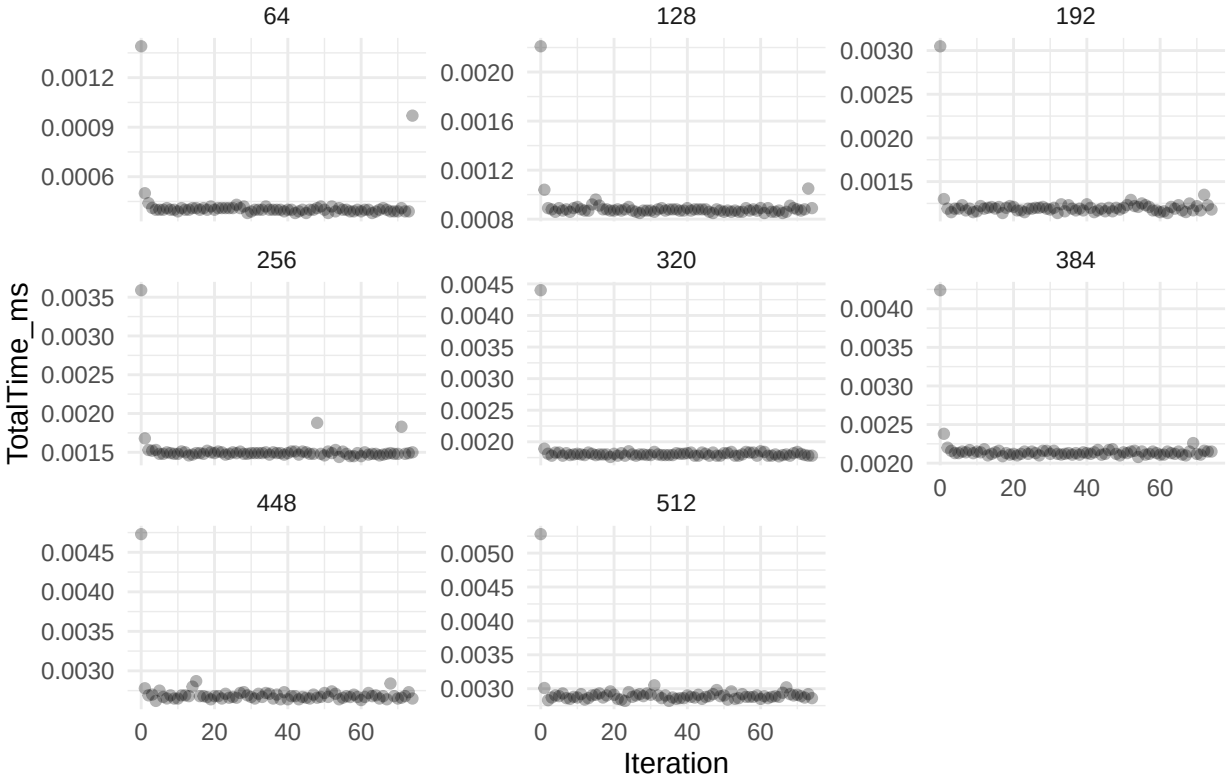

Warm−up Iterations for poisson−solver 1D, Double Prec, Full Periodic, CPU
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

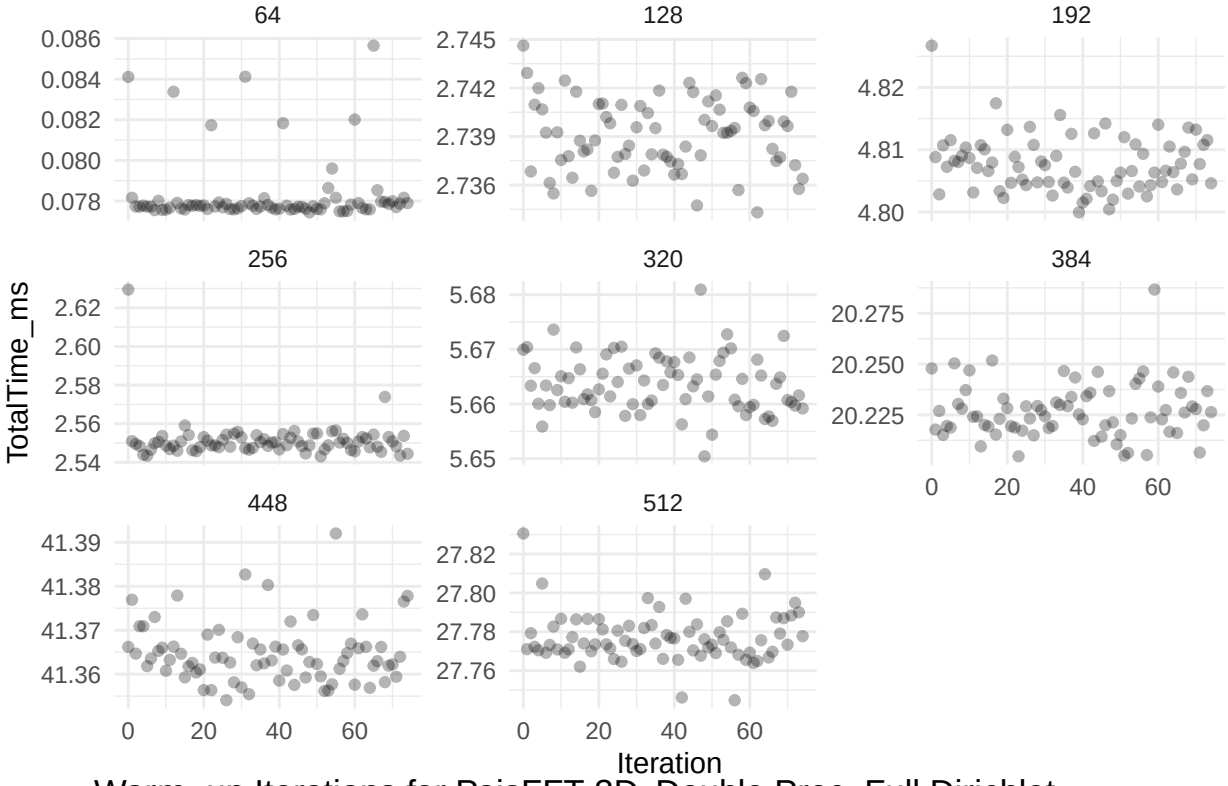

Warm−up Iterations for poisson−solver 2D, Float Prec, Full Neumann, CPU
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

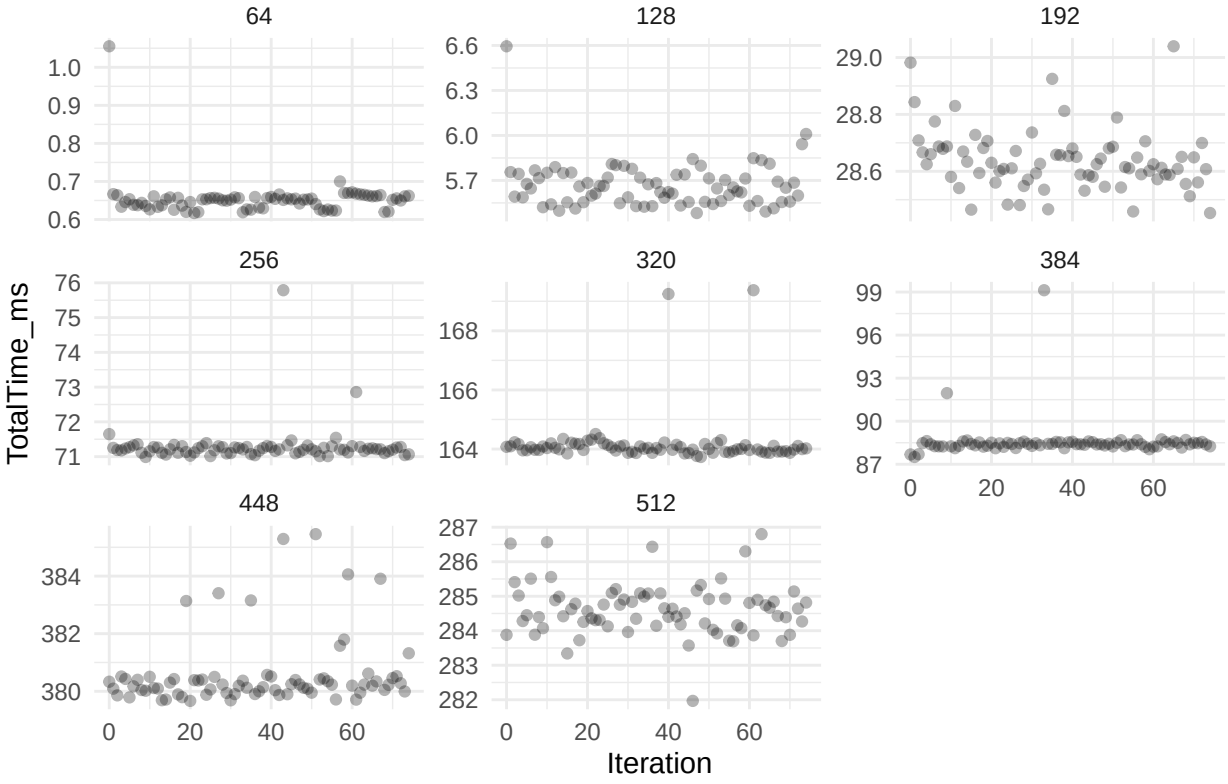

Warm−up Iterations for poisson−solver 3D, Double Prec, Full Dirichlet, G
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

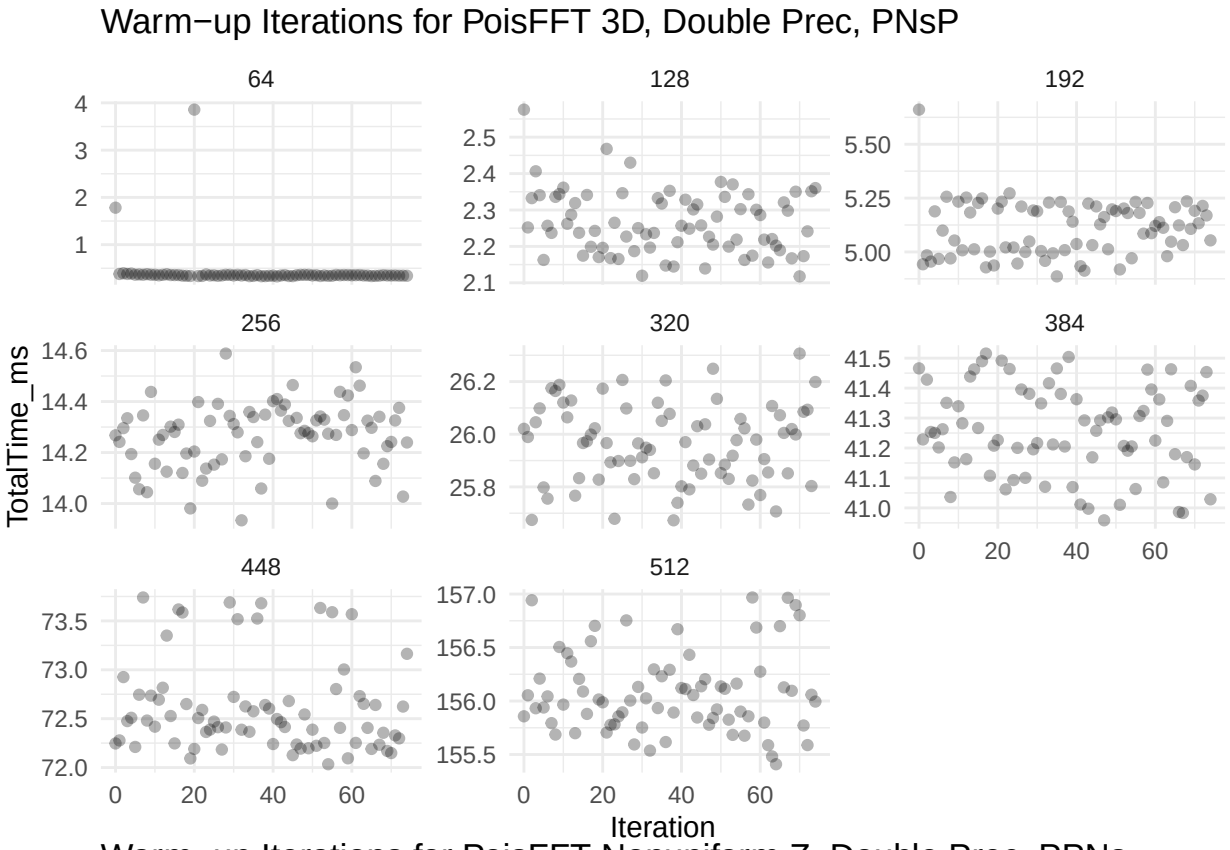

Warm−up Iterations for poisson−solver 3D, Double Prec, PNsP, CPU API
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration

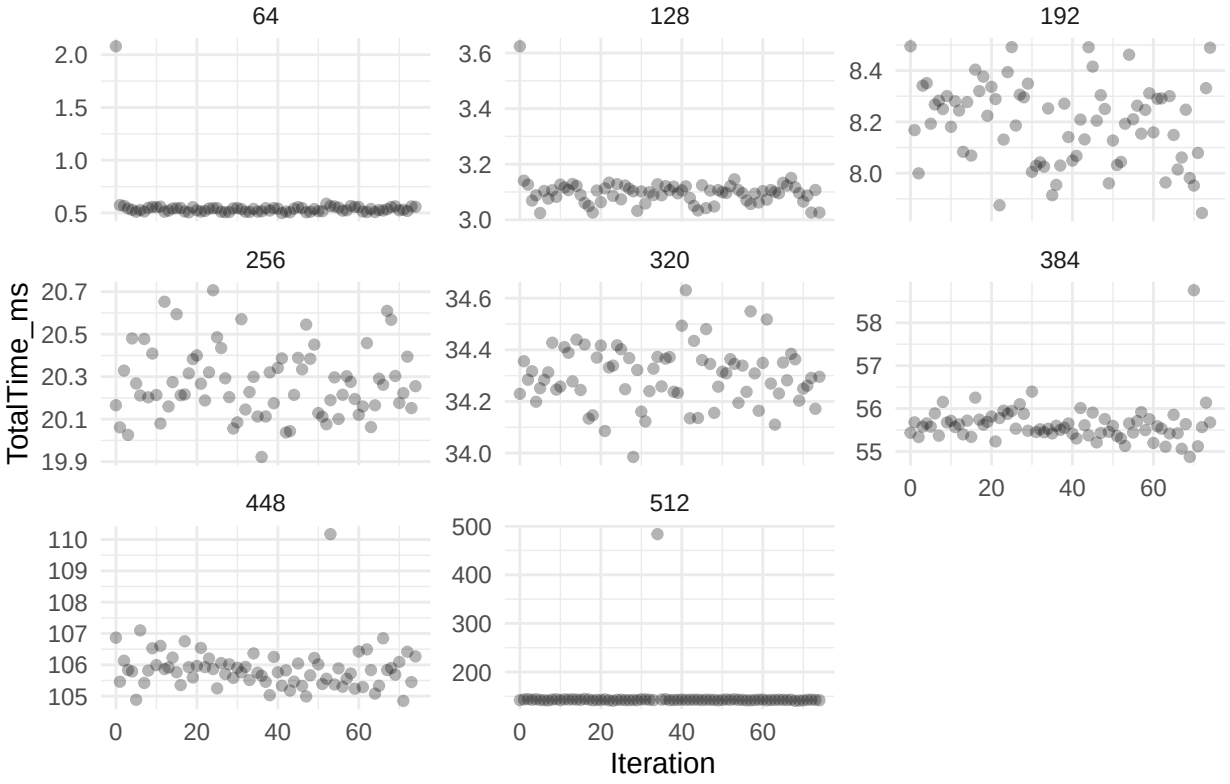

Warm−up Iterations for poisson−solver Nonuniform Z, Double Prec, PPNs
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration


Warm−up Iterations for PoisFFT 1D, Double Prec, Full Periodic
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration


Warm−up Iterations for PoisFFT 2D, Float Prec, Full Neumann
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration


Warm−up Iterations for PoisFFT 3D, Double Prec, Full Dirichlet
64
128
192
256
320
384
448
512
TotalTime_ms
Iteration




From the plots above it appears that the measurements have at most 1 to 2 warm-up iterations. We discard

those along with times greater than $3\sigma$.

### Finding Best `PoisFFT` Setup

For every `PoisFFT` configuration, we will select the number of cores that has the highest throughput average over all grid sizes.

### Comparison to `poisson-solver`

We will average the times for each grid size and compare `poisson-solver` against `PoisFFT`. The results are saved into the `plots` directory.

For all 1D grids, `PoisFFT` outperforms `poisson-solver`. For grids of this size, GPUs rarely provide any benefit over CPUs and this outcome is expected.

For 2D grids, the trend is that on grid side sizes of 64 `PoisFFT` outperforms `poisson-solver` (even the GPU API) but on larger grids, `poisson-solver` is faster. Here, we can also start to observe the bottleneck of our computation - buffer copy times and sometimes even FFTs.

For 3D grids, `poisson-solver` outperforms `PoisFFT` at all times when GPU API is used. For most boundary condition configurations, `poisson-solver` performs better than `PoisFFT`. However, `PoisFFT` achieves better performance than the CPU API of `poisson-solver` even for large grids for some boundary condition configurations. Examining the execution times of those configurations, we can notice that host-device buffer copying is a significant part of the bottleneck.

We can also observe that sometimes, FFT computation takes longer than buffer copying (e.g. for 3D full dirichlet configuration).

The table in `tables/local_speedup_table.html` lists speedups for the plotted configurations.

The speedups on 3D grids of `poisson-solver` CPU API compared to `PoisFFT` are up to `~20x`. For GPU API the speedups reach `~109x`.

For 2D grids the speedups are more impressive, reaching up to `~236x` for the CPU API and `~596x` for the GPU API.

### Conclusion for local solvers

While in many configurations `poisson-solver` (CPU API) provides significant speedup over `PoisFFT`, there still remain cases where the performance is identical or even worse. With manual examination, we confirmed that the buffer copy times between host and device are the main bottleneck, although FFT times are not to be ignored for certain configurations.

There is currently no other GPU FFT library that satisfies `poisson-solver` requirements. It may be worthwhile contributing to VkFFT to ensure maximum performance due to it currently being the only library to provide the same functionality as `FFTW` does on CPUs.

The copy bottleneck can be solved by converting more parts of the whole system to execute on the GPU and thus prevent the copies (or at least limit them to device-to-device copies which are shown to be much faster for `poisson-solver`).

Converting legacy Fortran code bases manually is not optimal as their code readability can be rather poor (with little to no documentation) and manual conversions thus become very time-consuming. The preferred solution would be to design a system for transforming selected parts of a Fortran code base automatically into CUDA, a solution which is currently being worked on.

### Distributed Solvers

TODO

# References

1. FUKA, V. PoisFFT – a free parallel fast poisson solver. *Applied Mathematics and Computation*. Online. 2015. Vol. 267, p. 356–364. DOI https://doi.org/10.1016/j.amc.2015.03.011.